

User guide for CALIBRATION tool (SENSEI)

5 de diciembre de 2025

Índice

1. Operating scheme	2
2. .inp file info_File containing the initial information	3
2.1. Intermediate .inp file intermediate_File	3
2.2. Time parameters	3
2.3. Demand patterns	4
2.4. Valve settings and initial status	4
3. .csv files for reading sensor measurements and/or pseudomeasurements	5
3.1. wlm_File, pm_File, fm_File and fm_avg_File files	5
3.2. dm_File file	6
4. Calibration options and options .csv file	7
4.1. Variables to optimize	7
4.2. Options .csv file (to activate variables, define groups of variables, and specify a chronology)	8
5. .csv files for specific calibration options	9
5.1. clusters_pipes_File file	9
5.2. common_DW_File file	10
5.3. common_HW_File file	11
5.4. main_pipes_File file	11
6. Aspects that affect calibration	12
6.1. Measurements / pseudomeasurements and weights	12
6.2. Jacobian	12
6.3. Iterative algorithm for calibration	13
6.4. Default values for algorithm parameters	15
6.5. Control of parameter increments	15
7. .txt configuration file (config_calibration.txt)	15
8. Calibrated model	16
9. .csv file with uncertainty propagation information	17

1. Operating scheme

This document explains how to configure the **SENSEI calibration tool**, which is applied to a water distribution network given a time series of measurements and/or pseudomeasurements over a certain period. Some specific details in the explanation correspond to the calibration exercise carried out on the *C-town* network.

It describes how to include measurement/pseudomeasurement `.csv` files, auxiliary files for calibration options, incompatibilities between options, exceptions to consider, and other preliminary requirements needed for the tool to function correctly. All necessary files are located in the **Calibration** folder. The user may enable/disable calibration options, modify values in the `.csv` files, add or remove measurements, etc., to observe how such changes affect the process.

The **Calibration** folder is structured as follows:

- **Executable** file: `Calibration_SENSEI.exe`
- **Configuration** file: `config_calibration.txt` (this is where the calibration is configured; the `.exe` will read and interpret it)
- Subfolder *C-town_Calib_Example*, which includes the following files:
 - **info_File**, named `C-town0.inp` (seed model for the calibration)
 - **dm_File**, **fm_File**, **pm_File** and **wlm_File**, named `dm_C-town.csv`, `fm_C-town.csv`, `pm_C-town.csv`, and `wlm_C-town.csv`, respectively (files containing measurements and/or pseudomeasurements, with their corresponding coefficients of variation)
 - **clusters_pipes_File**, **common_DW_File**, **common_HW_File**, **main_pipes_File**, named `clusters_pipes.csv`, `common_DW.csv`, `common_HW.csv`, `main_pipes.csv`, respectively (files required for certain calibration options)
 - **options_File**, named `options.csv` (file where groups of variables to be optimized are defined and the chronology that the process must follow is established, to approach the calibration via coordinate descent)
- 13 **dynamic libraries**: `epanet2.dll`, `libamd.dll`, `libcamd.dll`, `libccolamd.dll`, `libcholmod.dll`, `libcolamd.dll`, `libgcc_s_seh-1.dll`, `libgfortran-5.dll`, `libgomp-1.dll`, `libopenblas.dll`, `libquadmath-0.dll`, `libsuitesparseconfig.dll`, and `libwinpthread-1.dll`

The executable `Calibration_SENSEI.exe` opens a terminal through which the user can observe the progress of the calibration, which runs automatically based on the predefined configuration. The user simply needs to proceed through the pauses between each step by pressing any key when indicated.

Below are detailed the changes that the user is allowed to make and those that are prohibited.

What should the user NOT do?

- Delete any of the dynamic libraries (`.dll` files)
- Change the name of the configuration file `config_calibration.txt`, since the executable has been built to read a file with that exact name.
- Rearrange or, in general, modify the internal structure of the **Calibration** folder, as this may cause issues related to directories and file locations.

What can the user modify?

- Modify the content of the measurement or pseudomeasurement files (`dm_C-town.csv`, `fm_C-town.csv`, `pm_C-town.csv`, `wlm_C-town.csv`), changing existing values, removing or adding rows, etc.

- Modify the content of the `.csv` files used for certain calibration options.
- Modify the content of the initial `.inp` file (`C-town0.inp`), which will serve as the starting model for the calibration.
- Edit the options file (`options.csv`) to configure a different scheme for the calibration, in terms of the chronology and the way groups of variable types to be optimized are defined.
- Change the name of any file located in the *C-town_Calib_Example* subfolder (including the subfolder itself), as long as the new name is updated in the configuration file `config_calibration.txt`.
- Within `config_calibration.txt`, modify hyperparameters of the iterative calibration algorithm, although it is initially recommended to keep the default values.
- Select English or Spanish as the language for displaying information and messages during the process.

Nevertheless, **any modification must always be carried out following the guidelines indicated later on**, in each specific section.

2. `.inp` file `info_File` containing the initial information

The `.inp` file, which we will refer to as `info_File`, will serve as the starting model (**the algorithm's seed**) for the calibration, since it is used for an initial evaluation of the objective function. For this reason, in complex networks (either in terms of size or operation), the seed model may strongly influence the outcome. It is therefore recommended to begin with a plausible model or one that is somehow close to reality.

It is also used to store data related to the **topology** of the network, **count the number of elements** of each type, and **store key positions** of certain elements.

→ For more information on how to modify the `.inp` file in a way that is compatible with EPANET, **you can consult this [website](#)**.

2.1. Intermediate `.inp` file `intermediate_File`

The seed model `.inp` file `info_File` **will never be modified**. All changes applied to the network during the calibration process will be performed on an intermediate file, `intermediate_File`, which will be deleted once the process finishes (but not if it is interrupted halfway).

→ If the seed file is `info_File = C-town0.inp`, the intermediate file will be named `intermediate_File = C-town0_intermediate.inp`, and it will be located in the *C-town_Calib_Example* subfolder.

2.2. Time parameters

In the `[TIMES]` section of the `.inp info_File`, the user must specify several time-related parameters in the fields `DURATION`, `HYDRAULIC TIMESTEP`, and `PATTERN TIMESTEP`. Besides configuring the simulation parameters themselves, these values will also be used to define the dimensions of matrices and vectors required during the calibration. Therefore, it is important to understand this section well.

- **DURATION**: duration of the time series of measurements / pseudomeasurements. This will be the duration of each simulation performed.
- **HYDRAULIC TIMESTEP**: frequency for recalculating equations (as long as no event occurs earlier).
- **PATTERN TIMESTEP**: frequency of the collected data; or equivalently, the time interval between one data point and the next.

Important considerations for time parameters

- Once DURATION and PATTERN TIMESTEP are specified, the number of calibration periods T is determined as their quotient, so it is necessary that DURATION be a multiple of PATTERN TIMESTEP.

$$T = \frac{\text{DURATION}}{\text{PATTERN TIMESTEP}} = \text{number of periods for which data are available}$$

- Furthermore, it is advisable that HYDRAULIC TIMESTEP be a divisor of PATTERN TIMESTEP, although both parameters may also be equal.

For the C-town case, DURATION must be set to 167:00 (167 hours) and PATTERN TIMESTEP to 1:00, since the records have an hourly frequency over one week (excluding the last hour), which results in a calibration with $T = 167$ periods of one hour each. In addition, HYDRAULIC TIMESTEP has also been set to 1:00, which means the equations are recalculated every hour.

→ The hydraulic time step, HYDRAULIC TIMESTEP, influences the calibration results: the smaller the value of this parameter, the more realistic the approach (but also more computationally demanding). If the goal is to reduce simulation time and computational load as much as possible, one recommendation is to set HYDRAULIC TIMESTEP = PATTERN TIMESTEP.

2.3. Demand patterns

Demand patterns are a possible variable to optimize, which the user may enable if desired.

Requirement for demand patterns

Each demand pattern declared in the `info_File` must have length T.

- If any pattern has length $< T$, the user is warned to review it, and the process is not allowed to continue.
- If any pattern has length $> T$, only the first T multiplier factors will be considered.

→ EPANET does not enforce this condition: in a weekly simulation where patterns change every hour, one may define a 24-hour pattern (for example), and EPANET will repeat it each day of the week. If this is the case (or in a similar situation), the user should extend that *short* pattern as many times as needed to cover the T periods.

2.4. Valve settings and initial status

SENSEI also provides the possibility of optimizing valve settings. Below are the conditions that must be satisfied by the initial statuses of the valves, their settings, and the control rules associated with the valves that involve their setting.

Requirements for valve settings and initial status

About the setting

As usual, each valve is defined in the [VALVES] section of the `.inp` file. EPANET allows the setting defined in the [VALVES] section to differ from the setting imposed by a control (in the [CONTROLS] section). For example, the following combination is completely valid in EPANET:

```
[VALVES]
V2    J14    J422    254.0000    TCV    22.0000    0.0000
```

[CONTROLS]

LINK V2 15.0 IF NODE T2 BELOW 0.5000

However, **SENSEI** requires that the definition setting of a valve be equal to the setting imposed by any control rule that activates the valve (if such a rule exists).

About the initial status

A valve may start open, closed, or active (not to be confused *active* with *open*, which refers to *fully open*, as if it were a pipe). To specify the initial status, it is enough to write in the [STATUS] section the line 'V2 open' (fully open), 'V2 closed' (closed; does not allow water flow), or simply not include any line referring to V2 (which EPANET interprets as the valve starting the simulation active, operating according to its setting).

The issue is that, if the valve starts open or closed (i.e., if this has been specified in the [STATUS] section), SENSEI cannot correctly read the valve setting from [VALVES] using the EPANET toolkit. Therefore, if the valve actually begins closed (analogous for open), the user must make a prior modification in the `info_File`:

1. Remove the line 'V2 closed' from the [STATUS] section in the `.inp` file so that SENSEI can correctly read the setting.
2. Instead, **force it to start closed** using a time-based control in the [CONTROLS] section, which in this case must be 'LINK V2 closed AT TIME 0:00', and **it must appear before all other controls of the model (VERY IMPORTANT)**, so that it does not override the controls acting on that valve.

Thanks to this, SENSEI assumes that the valve is initially active (which allows it to read the setting), and as soon as the simulation begins, it becomes closed through the imposed control rule (until its status changes due to other controls, if any).

→ All these modifications required by SENSEI are due to limitations of the EPANET toolkit when interacting with valve characteristics.

3. .csv files for reading sensor measurements and/or pseudomeasurements

Although the structure of all these files is very similar, we distinguish two sections to highlight small differences.

3.1. wlm_File, pm_File, fm_File and fm_avg_File files

These files include the information collected by sensors measuring:

- Water level in tanks (NOT reservoirs *), in `wlm_File`.
- Pressure at junctions, in `pm_File`.
- Instantaneous flow, for those links equipped with sensors that measure flow at specific instants, in `fm_File`.
- Average flow during each period, for links with sensors that measure flow based on the accumulated volume over a time period, in `fm_avg_File`.

(*) For calibration, **SENSEI does not consider the piezometric head of reservoirs as a variable to optimize**. The user may choose whether to define a constant head for the reservoir (the most common in EPANET)

or to assign a pattern to account for changes over time; but in no case will the head given in the `info_File` be modified during the process, whether constant or pattern-based.

The files listed above are **optional**. If no sensors of a given type are available, it is not necessary to provide an empty `.csv` file.

The files must be structured as follows:

Structure of the sensor measurement `.csv` files

- Each row corresponds to one sensor.
- Regarding the columns:
 - 1st column: **ID of the element on which the sensor is installed**, which must match an element in the `info_File`.
 - For `wlm_File`, the ID must match that of a tank.
 - For `pm_File`, the ID must match that of a junction.
 - For `fm_File` and `fm_avg_File`, the ID must match that of a link (including pipes, pumps, and valves).
 - 2nd column: **coefficient of variation (CV) or measurement error rate**. This is the percentage by which the measurement may deviate from the real value. It must be entered as a number between 0 and 1.
 - In the remaining columns, the **measured values** must be provided, ordered by period. If any cell is left blank, it is interpreted that the variable is not measured at that instant/period (e.g., due to temporary sensor disconnection). Such “empty measurement” will be assigned zero weight.

→ EXTRA: if it is suspected that some measurements are less accurate at certain times, a **CV different from the general one may be specified in an extra row**, writing the different CV below the measurement to which it should be associated. For example:

	A	B	C	D	E	F	G	H
1	T1	0.01	3	2.82	2.7	2.62	2.7	2.89
2	T2	0.02	0.5	0.72	0.96	1.19	1.57	2.37
3					0.1	0.2	0.2	
4	T3	0.015	3	3.51	4.05	4.6	5.19	5.15

In the figure above, measurements from 3 tank-level sensors are shown for 6 time instants (corresponding to the start of each of the 6 intervals). Each of them has a different CV. For tank T2, a CV different from the general one is specified for the third, fourth, and fifth intervals.

3.2. `dm_File` file

The `dm_File` serves the same purpose as the files in the previous subsection; however, it must **include information on the average demand consumed in each interval by each demand node** (demand nodes are those junctions with nonzero base demand). This is not always feasible: sometimes not all demand nodes are equipped with a meter or sensor, or these sensors may not record data as frequently as those measuring other variables.

Demand pseudomeasurements

The `dm_File` must include **one row for each demand node**. If in practice this is not possible (due to lack of demand sensors or frequency), **pseudomeasurements** must be used:

- They are approximate values of the average consumed demand, based on **beliefs, historical records, or rough estimates**.
- To reflect the lack of accuracy of these approximations, it is recommended to assign a **relatively high CV**, for example between 0.1 and 0.3.

→ If, when reading `dm_File`, the number of rows does not match the number of demand nodes, an error message will be shown, requiring the user to exit and correct the `.csv` content.

4. Calibration options and options .csv file

4.1. Variables to optimize

SENSEI allows calibrating the network by considering as optimization variables both global parameters of various types and variables that affect each period (for example, demand patterns). The variables that the user may enable are the following:

1. **Demand patterns** over the T periods. This involves optimizing the modulation factor of each demand pattern at each period.
2. **Base demands** of the demand junctions (those whose base demand is nonzero)
3. **X-coordinates (flow)** of the pump curves
4. **Y-coordinates (head)** of the pump curves
5. **Valve settings** (with the specifics described earlier)
6. **Hazen–Williams roughness coefficient**,
 - either for each pipe in a given group of main pipes provided in `main_pipes_File`,
 - or for every pipe in the network.
7. **Darcy–Weisbach roughness coefficient common to each pipe group**. Each group must be defined through the `clusters_pipes_File`. Later, the conversion from a common coefficient R_{DW} to an individual coefficient R_{HW} (Hazen–Williams, which is the model used by SENSEI) for each pipe will be explained, based on its diameter. The initial D–W coefficient for each group must be specified in `common_DW_File`.
8. **Hazen–Williams roughness coefficient common to each pipe group**. Each group must be defined through the `clusters_pipes_File`. Unlike the previous point, here the H–W coefficient for each group is treated directly (without conversions), common to all pipes in the cluster. The initial coefficient must be introduced via `common_HW_File`.
9. **Initial tank levels** at the beginning of the series, since they influence the evolution of the simulation.
10. **Tank diameters**

→ **IMPORTANT:** SENSEI assumes that **pump curves have 3 points, of the form $\{(x_1, y_1), (x_2, y_2), (x_3, y_3)\}$** . Pump curves with a number of points different from 3 can be included in `info_File`, but as soon as one attempts to optimize any related variable (X-coordinates or Y-coordinates), an error will occur.

→ The **conversion from a common D–W coefficient for each group to an H–W coefficient for each pipe** is given by the following expression, in which the pipe diameter is involved:

$$R_{HW} = A + B (\log_{10} (R_{DW}) + 3)^C, \quad \varepsilon \text{ in mm} \quad (1)$$

where

$$\text{If } R_{DW} < 0,001 \Rightarrow R_{DW} = 0,001$$

$$A = 157 - 12e^{-5D}$$

$$B = -4,89 - 1,15 (\log_{10} D), \quad D \text{ (m)}$$

$$C = 1,97 - 0,263 (\log_{10} D)$$

The above expression applies to pipes whose diameter is defined in **mm** (i.e., in `.inp` files whose units are LPS or CMH).

The structure of the files `clusters_pipes_File`, `common_DW_File`, `common_HW_File`, and `main_pipes_File` will be explained in the next section.

4.2. Options .csv file (to activate variables, define groups of variables, and specify a chronology)

In the previous subsection, the different variables that the user may enable for calibration were detailed. Given the complexity of the problem in some situations, the user is allowed to choose which variables to activate, how to group them, and in which order. To do so, the stage at which each type of variable is to be calibrated must be specified in the `.csv options_File`, or the cell must be left empty if that option is not to be used.

Below is an example of the scheme used for the calibration of **C-town**. It consists of 6 stages: (1st) Demand patterns and base demands (both types of variables simultaneously), (2nd) Common H–W roughness coefficient for previously defined pipe groups, (3rd) Valve settings, (4th) X-coordinates of the pump curves, (5th) Y-coordinates of the pump curves, and (6th) Tank diameters.

	A	B
1	DEMAND PATTERNS	1
2	(INDIVIDUAL) BASE DEMANDS	1
3	PUMP CURVES X	4
4	PUMP CURVES Y	5
5	VALVE SETTINGS	3
6	INDIVIDUAL ROUGHNESS COEF.	
7	DW COMMON ROUGHNESS COEF (for a cluster)	
8	HW COMMON ROUGHNESS COEF (for a cluster)	2
9	INITIAL TANK LEVELS	
10	TANK DIAMETERS	6

Considerations regarding the `options_File`

- If calibration is to be performed for a given set of variables all at once, it is sufficient to assign a '1' to all variables that are to be optimized.
- The three options related to roughness are incompatible. If more than one of them is enabled, the user will be warned by a message to change it. The terminal will close and calibration will not proceed until the issue is resolved.
- It is essential **not to change the order of the rows in the `.csv options_File`**, since the executable `Calibration_SENSEI.exe` has been built to read the rows in the order shown in the figure.

→ Enabling a calibration option may require specifying the names of additional files in the configuration `.txt` file `config_calibration.txt`. For example, if the common H–W roughness option for a pipe group is activated, the corresponding filenames must be specified in that configuration file. All of this will be explained later, in a dedicated section for the configuration file.

5. `.csv` files for specific calibration options

As mentioned earlier, these are auxiliary files required when certain specific calibration options are enabled. They generally contain information that cannot be introduced in an `.inp` file. For example, it is not possible to define a pipe group in the `.inp`, which is why an auxiliary `.csv` file is required to store this information in specific structures created for that purpose. As before, if the user does not wish to optimize that type of variable, it is enough not to include it in the configuration file (there is no need to reference an empty `.csv` file).

5.1. `clusters_pipes_File` file

Considerations regarding the `clusters_pipes_File`

- Required if one wishes to use **either of the two approaches that work with a common roughness coefficient per pipe group**:
 - either Darcy–Weisbach (later converted into an individual H–W coefficient),
 - or Hazen–Williams.
- The `.csv` must consist of **2 columns**:
 1. Pipe ID, which must match the ID of some pipe in the `info_File`.
 2. Name of the group it belongs to (arbitrary, chosen by the user).

For example:

	A	B
1	P100	DMA1
2	P1000	DMA1
3	P1016	DMA2
4	P1025	DMA2
5	P398	DMA3
6	P376	DMA3
7	P11	DMA4
8	P141	DMA4
9	P399	DMA5
10	P1044	DMA5

→ Although in the example the pipes are grouped according to the DMA they belong to, this is not required; the grouping criteria may be different (for example, according to diameter ranges).

5.2. common_DW_File file

Considerations regarding the `common_DW_File`

- Complements `clusters_pipes_File` for the option of optimizing a common Darcy–Weisbach roughness coefficient for each group.
- It consists of **2 columns**:
 1. Group name, which must match some group defined in `clusters_pipes_File`.
 2. Darcy–Weisbach roughness coefficient (*mm*) for the group.

One possible option for the C-town case (if this option were to be enabled) would be to define the coefficients by DMA as follows:

	A	B
1	DMA1	0.97
2	DMA2	0.77
3	DMA3	0.65
4	DMA4	0.77
5	DMA5	0.65

This grouping is due to the fact that all pipes in the network are cast iron cement-mortar-lined pipes; and given that the average installation age of the pipes in DMA1 is 47 years, DMAs 2 and 4 is 27 years, and DMAs 3 and 5 is 15 years, the initial coefficient may be approximated by the expression $R_{DW_0} = 0,5 + 0,01 \cdot t$, where t is the age in years. However, this is only a rough initial approximation.

→ Before starting the calibration, and after reading the information and computing the H–W coefficient for each pipe (based on the D–W coefficient for each group), these new coefficients will be imposed on the initial model, and those provided in `info_File` will be ignored.

5.3. common_HW_File file

Considerations regarding the common_HW_File

- Complements the clusters_pipes_File for the **option of optimizing a common H–W roughness coefficient for each group**.
- It consists of **2 columns**:
 1. Group name, which must match some group defined in clusters_pipes_File.
 2. Hazen–Williams roughness coefficient for all pipes in the group.

An example may be:

	A	B
1	DMA1	120
2	DMA2	120
3	DMA3	120
4	DMA4	120
5	DMA5	120

→ Before starting the calibration and after reading the information, each pipe will be assigned the coefficient of its group in the initial model, and those originally given in info_File will be ignored.

5.4. main_pipes_File file

Considerations regarding the main_pipes_File

- This file is **optional** when the **option of individually optimizing the H–W roughness coefficient of pipes** is enabled.
 - If included ⇒ it defines a group of pipes (intended to be the main ones) whose roughness will be optimized. The roughness of the remaining pipes will be kept as in the seed .inp file.
 - If not included ⇒ the individual optimization of the roughness of all pipes in the network will be performed.
- It consists of **a single column**, in which the ID of the pipes whose roughness is to be optimized must be specified (they must match pipe IDs in info_File).

For example,

	A
1	P310
2	P98
3	P97
4	P83
5	P100
6	P22

6. Aspects that affect calibration

This section provides a brief description of the behaviour of certain components of the SENSEI calibration process, giving the user a general idea of how the choice of specific parameters or *inputs* may influence the final calibration result. In addition, it will be indicated how to modify some of these “ingredients” through the configuration file.

6.1. Measurements / pseudomeasurements and weights

The reading of the measurement and pseudomeasurement `.csv` files provides, for each defined element and at each instant, a measurement z_i with an associated coefficient of variation CV_i . This allows us to define a standard deviation of the measurement/pseudomeasurement as $CV_i \cdot z_i$. The corresponding weight of z_i , denoted by w_i , is defined as the inverse of the measurement variance:

$$w_i = \frac{1}{(CV_i \cdot z_i)^2} \quad (2)$$

Nuances regarding the weights

- The lower the variability of a measurement, the greater our confidence in it and the larger its weight; and vice versa. The weight (relative to the measured value) indicates the *importance* that the variable will have in the objective function.
- If $z_i = 0$ (the measurement is zero; for example, a flow sensor reading zero because no water is passing through the pipe), then $w_i = 0.05$ automatically, to avoid division by zero in (2).
- If a variable is not measured at some time (i.e., the corresponding cell in the `.csv` file is empty), it is assigned zero weight, $w_i = 0$, so that it does not influence the objective function.

6.2. Jacobian

The Jacobian matrix is the key component of the optimization algorithm. It contains the derivatives of the measurements (*rows*) with respect to the variables (*columns*) modified at each defined stage.

Numerical computation of the Jacobian

Some derivatives, depending on the parameter or variable with respect to which they are computed, have analytical expressions that are easy to evaluate; but for others this becomes more complicated. Therefore, **SENSEI computes the derivatives numerically**:

- At each iteration, the basis is the vector $g(\mathbf{x})$, which contains the values of the measurements/pseudo-measurements computed via simulation for a set of variables and parameters $\mathbf{x} = (x_1, \dots, x_n)$.
- The variable with respect to which the derivative is to be computed (say the j -th variable) is perturbed by $\varepsilon\%$, giving the perturbed vector $\mathbf{x}_\varepsilon = (x_1, \dots, x_j(1 + \varepsilon/100), \dots, x_n)$. A new simulation produces the vector $g(\mathbf{x}_\varepsilon)$.
- The derivative vector (the entire j -th column) is then computed as:

$$\frac{g(\mathbf{x}_\varepsilon) - g(\mathbf{x})}{x_j \cdot \varepsilon/100} \quad (3)$$

→ For the numerical derivatives computed via (3) to be faithful to the model, the perturbation must be *small*. Therefore, the default perturbation rate is $\varepsilon = 0.2$, though it will be explained later how to change it.

6.3. Iterative algorithm for calibration

Reading the `options_File` provides the number of stages, `num_stages`, in which the calibration will be carried out. SENSEI will sequentially loop through these stages up to a maximum of `max_loops` times.

Within each stage, all variables and parameters are kept fixed except the ones being optimized at that stage. We iterate up to `max_it` times before moving to the next group of variables.

In each stage, we solve the **Levenberg–Marquardt** equation:

$$(J_k^t \cdot W \cdot J_k + \lambda \cdot D_k^t \cdot D_k) \cdot \Delta \mathbf{x}_{k+1} = J_k^t \cdot W \cdot (z - g(\mathbf{x}_k)), \quad (4)$$

where $D_k^t \cdot D_k$ is a diagonal matrix whose diagonal contains the entries of $J_k^t \cdot W \cdot J_k$.

→ A **regularization term** λ is included. For each stage, SENSEI begins with an initial value λ_0 and:

- If on the first attempt (first solution of (4) within the stage) the objective function does not improve, λ is increased (more regularization) and (4) is resolved. This continues up to `max_attempts`, after which the algorithm abandons the stage and proceeds to the next.
- Otherwise, if the objective function improves, the variables are updated with the computed values, λ is decreased (less regularization), and optimization continues for that group of variables until convergence (within the stage) or until `max_it` iterations are reached.

Algorithm pseudocode

```

1 x0 ← initial status of the network    # initial values of parameters and
   variables
2 Compute residuals r = z - g(x0)
3 F(x_prov) = r^t*W*r    # evaluate objective function for seed variables
4 SWSE ← F(x0)    # initial objective function value
5
6 for loop in 1:max_loops:
7     for stage in 1:num_stages:
8         lambda ← lambda_init    # initialize lambda in each stage
9         for it in 1:max_it:
10            attempts ← 0

```

```

11         break_while ← 0
12         x_k ← current status
13
14         Compute J(x_k)
15
16         while break_while == 0:
17             Solve: (Jt*W*J + λ*Dt*D) Δx = Jt*W*r
18             x_prov ← x_k + Δx
19             Compute residuals r = z - g(x_prov)
20             F(x_prov) = rt*W*r
21             SWSE_prov ← F(x_prov)
22
23             if SWSE_prov < SWSE:
24                 x_{k+1} ← x_prov
25                 SWSE ← SWSE_prov
26                 lambda ← lambda / 10
27
28                 if convergence:
29                     break_while ← 1
30             else:
31                 lambda ← 10 * lambda
32                 attempts += 1
33                 if attempts > max_attempts:
34                     break_while ← 1

```

Note: $SWSE = \text{Sum of Weighted Squared Errors}$

Let $\mathbf{x}_{\text{new}}$ and $\mathbf{x}_{\text{old}}$ be the newly updated variables and the previous ones, respectively. Convergence is evaluated by:

$$\frac{\|g(\mathbf{x}_{\text{new}}) - g(\mathbf{x}_{\text{old}})\|_1}{\|g(\mathbf{x}_{\text{new}})\|_1}$$

If the ratio is less than the tolerance `tol`, the algorithm is considered to have converged and proceeds to the next stage.

At the beginning of each loop, the change between $g(\mathbf{x}_{\text{loop}})$ and its equivalent from the previous loop, $g(\mathbf{x}_{\text{loop}-1})$, is evaluated. If the ratio is below `tol`, the calibration process stops, as it is considered that the algorithm has converged and further improvement is unlikely.

`.inp` file storing the latest update

Each time the algorithm enters a new stage, an `.inp` file is saved with the current parameter and variable values. If `info_File = C-town0.inp`, the file will be named `LATEST_IMPROVEMENT_C-town0.inp`, since it contains the latest stored update. It will be located in the same folder as `info_File` (in our case, ***C-town_Calib_Example***). This file is useful as a saved copy of the latest update in case execution problems arise. At the end of calibration (but not if aborted early), the file is deleted automatically (the final calibrated model is stored in a specific file, described later).

6.4. Default values for algorithm parameters

Default values for algorithm parameters

- Perturbation rate for computing the Jacobian: $\varepsilon = 0.2$
- Initial value of the regularization parameter in each stage: $\lambda_0 = 0.01$
- Maximum number of times the calibration scheme is looped: `max_loops = 20`
- Maximum number of iterations within each stage: `max_it = 5`
- Maximum number of attempts increasing λ (regularization) when solving (4): `max_attempts = 5`
- Tolerance: `tol = 1e-5`

→ The calibration parameters can be modified in the configuration file (explained in its specific section), but **it is recommended to keep the default values** at least for the first run.

6.5. Control of parameter increments

Solving (4) provides the increments of the optimized parameters, $\Delta \mathbf{x}_{k+1}$. The provisional variables are computed by adding these increments to the current values:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta \mathbf{x}_{k+1} \quad (5)$$

It may happen that if the problem is ill-conditioned or the optimization hyperparameters are not appropriate, some of the provisional variables in (5) become physically impossible or undesirable due to being implausible. Examples include negative demands (when the node is not a supply node), pump curves for which the points $(x_1, y_1), (x_2, y_2), (x_3, y_3)$ violate basic rules (e.g., $x_3 < x_2$), extreme roughness coefficients, etc.

7. .txt configuration file (config_calibration.txt)

In this file, named `config_calibration.txt` and **whose name cannot be changed**, the user must specify the language in which information and messages will be displayed, the name and location of the files used for calibration, and the values of the algorithm's hyperparameters. An example is:

```
1 LANGUAGE (ENG/ESP); ENG
2 -----
3 PATH; C-town_Calib_Example
4 -----
5 info_File; C-town0.inp
6 dm_File; dm_C-town.csv
7 wlm_File; wlm_C-town.csv
8 pm_File; pm_C-town.csv
9 fm_File; fm_C-town.csv
10 fm_avg_File; -
11 clusters_pipes_File; clusters_pipes.csv
12 common_HW_File; common_HW.csv
13 common_DW_File; common_DW.csv
14 main_pipes_File; main_pipes.csv
15 options_File; options.csv
16 -----
```

```

17 epsilon; -
18 lambda_init; -
19 max_loops; -
20 max_it; -
21 max_attempts; -
22 tol; -

```

Considerations regarding the configuration .txt file

- The **language** may be **English or Spanish**, activated by specifying '*ENG*' or '*ESP*', respectively, in the **LANGUAGE**. If the value is left empty or anything else is written, English will be activated as the default language.
- It is **essential** that the .csv files (measurement/pseudomeasurement files and extra calibration files) and the .inp info_File be located in a **subfolder**, indicated in **PATH**; in our case *C-town_Calib_Example*.
- It is **mandatory to specify an initial model info_File** and a **measurement/pseudomeasurement file for the demands** in all demand junctions, **dm_File**.
- It is **not necessary to include** wlm_File, pm_File, fm_File, or fm_avg_File if those measurements are not available. In such cases, one must write ' - ' (a **dash, NEVER leave it blank**), as shown in the example above for fm_avg_File.
- The same applies to extra calibration files, if their corresponding options have not been activated.
- To use the **default values for the algorithm's hyperparameters**, simply write ' - ' (**dash**), as in the example. If any value is to be modified (not recommended at first), it must be specified explicitly.
- **IMPORTANT: Do not change** the text on the left-hand side (before the ' ; '), because the executable was built to read it exactly as written in order to store the information properly.

→ When the **option to optimize the H–W roughness coefficient of pipes individually** is activated, whether the optimization is performed **for a specific group of pipes or for all pipes** depends on whether a **file name** is provided for **main_pipes_File** in the configuration .txt file.

→ If the configuration file includes names of extra files corresponding to calibration options that have **not** been activated in **options_File**, these files will be ignored, as they are not required for the defined calibration. A warning message will be displayed in the terminal to notify the user, in case they forgot to activate the option associated with that file.

8. Calibrated model

The algorithm may exit the main loop for the following reasons:

- The set of activated variables has been cycled through **max_loops** times.
- Convergence has been achieved because the change in $g(\mathbf{x})$ between one *loop* and the next is smaller than the tolerance **tol**.

In such cases, a **solution file containing the calibrated model** is produced. The name of this file is based on the name of **info_File** and includes the date and time when it was saved (to avoid overwriting previously calibrated models), in formats **YYYYMMDD** and **hhmmss**, respectively. For example, if **info_File** is named *C-town0.inp* and calibration finishes on 24/12/2025 at 21:35:42, the calibrated model file will be named

`C-town0_calibrated_model_20251224_213542.inp`, and it will appear in the same **subfolder** (in our case, *C-town_Calib_Example*).

If the initial point is far from the solution, if the seed model is not close to reality, or if measurements are scarce, convergence may become “stuck” far from a satisfactory solution (local minimum). To address this issue, several strategies are proposed to help reorient the calibration process.

Tips for achieving algorithm convergence

Remember that `C-town0_calibrated_model_YYYYMMDD_hhmmss.inp` is, up to this point, the best solution found for the defined problem (given the provided files and chosen configuration). The user may run `Calibration_SENSEI.c` again after:

- having set `C-town0_calibrated_model_YYYYMMDD_hhmmss.inp` as the `info_File` (that is, **using the best solution found as the new seed model**)
- and having made one of the following changes:
 1. Modify the configuration of optimization variables in `options_File`: regroup them differently, activate new variables, change the order, etc.
 2. Change hyperparameters of the calibration algorithm: `max_loops`, `max_it`, `max_attempts`.
 3. Change the initial value of the regularization parameter λ in (4).
 4. Change the perturbation rate ε used to compute the Jacobian derivatives.

REMEMBER that the solution network (calibrated model) is the **most probable parameter configuration** given the provided measurements/pseudomeasurements. **It may not be an exact reflection of reality** if factors such as topology, the initial model `info_File`, or the collected measurements are inadequate. The higher the quality of these “ingredients”, the more realistic the solution provided by *SENSEI* will be.

9. .csv file with uncertainty propagation information

When the loop has been exited (either due to convergence or because `max_loops` has been reached), *SENSEI* considers all variables that were optimized and computes the Jacobian using all of them simultaneously.

It then propagates the uncertainty, thus obtaining a standard deviation for the adjusted value of each optimized variable. With this, the posterior coefficient of variation of each parameter or variable is computed.

All this information is stored in a `.csv` file located in the same folder as `info_File`. In our case, the folder is *C-town_Calib_Example*, and if `info_File` is named `C-town0.inp`, the `.csv` file will be named

`C-town0_propagation_uncertainty_YYYYMMDD_hhmmss.inp`

The timestamp will be slightly later than the time at which the calibrated model `.inp` was saved, since the uncertainty propagation process occurs afterwards.